PromoPilot

Reads microdrama screenplays, finds the promo moments worth posting, and ranks them, with every pick checked in code.

Kuku AI Generalist assignment · **Problem 1: Find the Promos**
Live app: **Streamlit app** · Code: **github.com/shobhit0521/Kuku_assignment**

> **The key idea: the model points, the code measures.** Gemini makes the judgement calls (which moment grips, which line hooks). It can only *point* at screenplay lines by ID, and Python computes every timestamp and enforces every rule. So the tool cannot invent a moment or break a rule, and every rejection comes with a reason.

50	147	37	110	25
episodes read, the full input pack	candidate clips proposed and checked	rejected by the checks, each with a reason	passed every check (full catalogue)	ranked promos in posting order

1 · THE PROBLEM AND WHY I PICKED IT

An editor watches every episode and picks promo moments by hand, and Kuku releases new shows every week, so the manual step does not scale. I picked Problem 1 because the input is concrete and **most of the output is checkable**: a clip is either 20–60 s or it is not, and the hook either lands in the first 3 s or it does not. That let me split the work cleanly: rules a computer can verify, and taste a model has to judge.

2 · ASSUMPTIONS WHERE THE BRIEF WAS SILENT

- **Screenplay times match the final cut**, so clip timestamps transfer directly to the video.
- **One continuous segment per promo.** Stitched promos are out of scope for this version (see §8).
- **Hook** = a line starting ≤ 3 s into the clip. **Cliffhanger** = the last story beat, with ≤ 2 s after it, left unresolved.
- **A clip never cuts through a line of dialogue**, and never ends on a bare transition.
- **"Makes sense on its own"** is tested by a judge that sees only the clip, never the episode.
- **A posting feed needs variety:** at most one promo per episode and three per story type.

3 · HOW IT WORKS

Python, deterministic · AI model call

Parse — Scenes and timed lines, each with an ID (EP001_B027) › **Generate** — Gemini 3.5 Flash proposes ≤ 5 clips per episode, as line IDs only › **Validate** — Hard rules; one automatic repair › **Blind judge** — Flash Lite scores 6 dimensions, citing lines › **Check + rank** — Verify citations, re-run rules, gates, rank

- **Hard rules (code, final):** 20–60 s, hook ≤ 3 s, no dialogue cut, no transition-only ending, cut ≤ 2 s after the suspense beat. A failing clip gets one repair (e.g. extend to finish a line) and is checked again from scratch. The judge cannot override a hard-rule failure.
- **Blind judge:** sees only the clip's lines under neutral IDs, never the generator's reasoning, so it cannot just agree with it and it reads like a first-time viewer. Every score must cite lines; Python verifies each citation.
- **Quality gates:** hook, cliffhanger and stand-alone context ≥ 3/5, and the ending must be unresolved.
- **The judge is checked too:** if a score contradicts the judge's own notes, it gets one reassessment. If the contradiction remains, the original is kept and the clip is flagged for a human.

4 · WHO DOES WHAT

AI (Gemini)	Code (Python)	Human (editor)
Proposes clips as line IDs	Owns every timestamp and duration	Approves the shortlist before posting
Scores clips blind, citing lines	Enforces hard rules and gates	Decides the 3 flagged clips
Writes the one-line pitch	Verifies citations, ranks, removes near-duplicates	Sets the weights and gates

5 · RANKING: WHICH TO POST FIRST, AND WHY

Score = hook 30% + stand-alone context 30% + cliffhanger 25% + emotional impact 15%. A promo is shown to strangers, so **stopping the scroll** and **making sense cold** weigh most; the cliffhanger is what turns a view into a tap. Ties go to the stronger cliffhanger, then hook, then context. The shortlist then keeps the feed varied: one promo per episode, ≤ 3 per story type, and clips overlapping > 50% removed.

Post first: #1 · Episode 1 · score 5.00 of 5

Segment	00:39.00 → 01:18.50 (39.5 s, one continuous stretch)
Hook	00:39.10 · "*Because he is marrying me.*" (0.1 s in)
Cliffhanger	01:15.00 · "*Long enough to know he was never going to marry you.*"
Why it works	"Witness the ultimate betrayal when a sister steals the bride's groom at the altar." (tool's line)
Why first	Top score on every weighted dimension, and it comes from Episode 1, so a viewer who taps lands at the start of the show.

#	Clip	Segment	Score	Opens on (hook)
1	EP001_C03	00:39.0–01:18.5	5.00	"Because he is marrying me."
2	EP011_C01	00:16.8–00:53.6	4.60	"Your husband stole my gold pocket watch!"
3	EP028_C02	00:37.7–01:10.2	4.55	"I was poisoned!"
4	EP031_C02	00:10.5–00:58.8	4.55	"The great Lady Juliana Ashcroft."
5	EP002_C02	00:00.0–00:44.7	4.45	"I cannot marry you, Juliana."

All 25 promos, with hook and cliffhanger timestamps, verbatim lines and the reason for each, are in `shortlist.csv` and the live app. Every promo is one segment, 20.7–57.5 s.

6 · THE PROTOTYPE

The **Top promos** page shows exactly what the brief asks for, plus where the tool picks badly; the sidebar holds every clip's evidence and rejections. **Analyse your own screenplays** runs the same pipeline live on uploaded `.txt` or `.zip` episodes (the interview re-run). **The model is a setting:** one OpenAI-compatible client, so a stronger model plugs in through environment variables.

7 · HOW I KNOW IT'S GOOD

Outcome of all 147 clips	Clips
Eligible: passed every hard rule and quality gate	**110**
Too weak to post: failed a quality gate (cliffhanger < 3 ×19, context < 3 ×13, ending resolved ×8, hook < 3 ×1)	28
Broke a hard rule: cut a line of dialogue (4), ran on too long after the suspense beat (5)	9
Not judged or invalid judge output	0

- **Rules are facts, not opinions.** Every shortlisted promo is verified in code against the screenplay's own timestamps. No moment is invented, because the model can only cite existing line IDs.
- **Automatic repair worked:** 6 clips that broke a rule were repaired and re-judged from scratch, and 5 became eligible.
- **No forced picks:** in EP006 and EP038 every candidate failed a gate, so the tool reports "no strong promo" instead of filling the slot. 48 of 50 episodes have one.
- **The ranking is stable:** re-ranking with equal weights keeps 24 of the 25 shortlisted promos, with a rank correlation of 0.97 across all 110 eligible clips.
- **The judge's contradictions are caught:** 33 reassessments were run, and 3 clips stayed contradictory and are flagged for a human (none is shortlisted).
- **413 automated tests** cover the parser, the rules, repairs, the judge checks, the ranking and the app.

8 · WHERE IT PICKS BADLY

- **AI judged AI.** The scores are one model's opinion, from the same family as the generator. They cluster at 3–4 (median 3.7), a 5.0 is probably lenient, and gaps under ~0.5 are noise. There is no human-labelled set and no baseline comparison yet, so I claim no accuracy figure.
- **#2 depends on the weights:** with equal weights, EP011_C01 drops out and EP043_C01 takes its place.
- **The one-line pitches read like ad copy** and are unreviewed. #2's is a question ("Will the framed husband's dangerous calm expose the accuser…?"), not a reason.
- **Recaps:** clips that start at 00:00, such as #5 (EP002_C02), may open on a recap of the previous episode, which a script-only judge cannot tell apart.
- **Camera beats:** a suspense beat can be a camera shot that the clip cuts before it finishes (EP003_C02).
- **Early episodes dominate:** EP001 has 4 eligible clips, two of them overlapping. The diversity rules fix the shortlist, not the catalogue.
- **Single segments only:** no promos stitched from several moments or episodes.
- **Screenplay, not footage:** it cannot judge acting, pacing or music.

9 · SCALE AND COST, FOR EVERY SHOW KUKU MAKES

- **Per episode:** 1 generation call and about 3–4 judge calls, roughly **23k tokens** and **38 s** of model time. The whole 50-episode show used ≈ 1.17 M live tokens (0.53 M generation, of which 0.26 M was hidden reasoning, and 0.64 M judging), with 31 min of model time in total.
- **Cost per show** = tokens × the chosen model's price, growing linearly with episodes. Episodes are independent, so with more quota they can run in parallel instead of one after another.
- **Robust at volume:** every model reply is cached (re-runs are free), runs resume after errors, request budgets cap spending, and a quota error stops cleanly. In this run, 6 server errors (HTTP 503) were retried and one quota stop (HTTP 429) was resumed with nothing lost.
- **Free tier, honestly:** the run hit Gemini's free limit of 20 Flash requests a day, so I enabled billing on the same project to finish within the deadline. Nothing paid-only is used; on the free tier the same run just takes several daily resets.
- **Editor time** shifts from watching 50–100 minutes of footage to reviewing 25 cards with evidence. A new show is just a new input folder and run name.

10 · AI TOOLS USED, AND ONE PLACE THEY GOT IT WRONG

Tools: Cursor (AI coding assistant) to build, test and review the code; Gemini 3.5 Flash (generator) and Gemini 3.5 Flash Lite (blind judge) at runtime, through Gemini's OpenAI-compatible API.

Where AI got it wrong: for EP036_C03 the generator named a hook line that starts **3.9 s** into its own clip, breaking the 3-second rule it had been given. Because models only cite line IDs and Python computes every offset, the validator caught it at once and marked the clip as failing. It became eligible only after the blind judge, which never saw that choice, picked a line **1.1 s** in and Python re-checked the rule. The judge was wrong too: in 3 reassessments it simply *raised* a disputed score instead of fixing the contradiction. Python rejected those changes and flagged the clips. **This is why the rules live in code, not in the prompt.**

11 · HOW I'D KNOW IT WORKS WITH A REAL TEAM

- **Editor agreement:** log each accept or reject, the share posted unchanged, and boundary edits.
- **Audience results:** 3-second hold rate (did the hook work?), watch-through, and taps into the app (did the cliffhanger work?).
- **Close the loop:** those labels become the ground truth that calibrates the judge and the weights.

12 · WHAT I'D BUILD NEXT

- A **hand-checked set** (e.g. 20 picks and rejections) to measure how often the judge agrees with an editor.
- A **naive baseline** ("the last 30 s of each episode") scored by the same judge, to prove the tool beats the obvious. The code for it exists; it has not been run.
- **Stitched promos** joining 2–3 moments, reusing the same rule checks per segment.
- A **judge from a different model family**, for a more independent second opinion.

All figures are from run `fullpack-v1-20261106` (6 Nov 2026), whose unedited outputs are in the raw-outputs folder. Scores are an AI model's assessment, not measured audience performance.